



GUÍA DE ESTUDIO

Plataforma Acme School

Análisis Completo de Arquitectura y Código

01 Módulo de Login

Web Components · Shadow DOM · ES Modules

02 Portal de Creación

Clases · localStorage · DOM Dinámico

03 Resolver Examen

URLSearchParams · setInterval · Resultados

Conceptos Clave

4 Temas Teóricos

Desglose Práctico

8 Funciones Críticas

Troubleshooting

6 Errores Comunes

Simulacro

5 Preguntas de Examen

ÍNDICE DE CONTENIDOS

1.	ARQUITECTURA GENERAL DEL PROYECTO	3
	1.1 Visión general de módulos	3
	1.2 Flujo de datos (localStorage)	3
2.	CONCEPTOS CLAVE	4
	2.1 Web Components y Shadow DOM	4
	2.2 ES6 Módulos (import / export)	5
	2.3 Clases en JavaScript (OOP)	5
	2.4 localStorage — CRUD completo	6
3.	DESGLOSE PRÁCTICO	7
	3.1 auth.js — Autenticación	7
	3.2 login.js — Web Component de Login	8
	3.3 ExamenStorage — Clase estática	9
	3.4 GestorExamenes — Gestión de UI	10
	3.5 recolectarPreguntas()	11
	3.6 iniciarTemporizador()	12
	3.7 finalizarExamen()	13
	3.8 previewExamen.js — URLSearchParams	14
4.	TROUBLESHOOTING	15
5.	SIMULACRO DE EXAMEN	17

SECCIÓN 01

Arquitectura General del Proyecto

Acme School es una plataforma educativa compuesta por **tres módulos** independientes que se comunican exclusivamente a través de **localStorage** del navegador, sin backend ni base de datos externa. Cada módulo es una carpeta HTML/CSS/JS que intercambia datos mediante claves compartidas en el Storage del navegador.

Módulo	Archivos Principales	Tecnología Destacada	Clave localStorage
Login	index.html, login.js, auth.js	Web Component, Shadow DOM, ES Modules	"usuarios", "sesion"
Creación de Exámenes	CrearExamen.html, app.js	Clases estáticas, Event Delegation, insertAdjacentHTML	"exámenes_data"
Resolver Examen	Examenes.html, PreviewExamen.html, RealizarExamen.html + JS	URLSearchParams, setInterval, Web Component	"examen_actual", "resultados_examenes"

1.2 Flujo de Datos

El flujo de navegación entre páginas sigue siempre la misma dirección:

```
index.html (Login)
  → Valida credenciales en localStorage["usuarios"]
  → Guarda sesión en localStorage["sesion"]
  → Redirige a CrearExamen.html

CrearExamen.html (Admin)
  → Lee / escribe exámenes en localStorage["exámenes_data"]
  → Navega a Examenes.html para vista pública

Examenes.html (Público)
  → Lee localStorage["exámenes_data"] para listar exámenes
  → Redirige a PreviewExamen.html?codigo=EX-001

PreviewExamen.html
  → Lee ?codigo de la URL (URLSearchParams)
  → Guarda sesión del estudiante en localStorage["examen_actual"]
  → Redirige a RealizarExamen.html

RealizarExamen.html
  → Lee localStorage["examen_actual"]
  → Al terminar, guarda resultado en localStorage["resultados_examenes"]
```

IMPORTANTE

TODOS los módulos comparten el mismo namespace de localStorage del navegador. Si el usuario borra los datos del navegador, toda la información se pierde. Esta arquitectura es apropiada para prototipado pero no para producción.

SECCIÓN 02

Conceptos Clave

2.1 Web Components y Shadow DOM

Un **Web Component** es un elemento HTML personalizado que encapsula su propia lógica y estilos. En este proyecto se usan dos: **<login-acme>** (login.js) y **<resolver-examen>** (RealizarExamen.js). Ambos utilizan **Shadow DOM**, un árbol DOM aislado que impide que los estilos externos interfieran con el componente.

Partes esenciales de un Web Component:

API	Descripción
class X extends HTMLElement	Hereda el comportamiento de un elemento HTML nativo
super()	Llama al constructor del padre (HTMLElement). Siempre va primero en el constructor.
this.attachShadow({mode:"open"})	Crea el Shadow DOM. "open" permite acceder desde JS externo con element.shadowRoot
connectedCallback()	Lifecycle hook: se ejecuta cuando el elemento es insertado en el documento
this.shadowRoot	Referencia al DOM aislado. Usar shadowRoot.getElementById() en vez de document.getElementById()
customElements.define("tag", Clase)	Registra la clase como elemento HTML personalizado con el tag indicado

```
// Estructura base de cualquier Web Component del proyecto
class LoginAcme extends HTMLElement {
  constructor() {
    super(); // SIEMPRE primero
    this.attachShadow({ mode: "open" }); // Crea el Shadow DOM
  }
  connectedCallback() {
    // El HTML del <template> se clona y se inserta en el Shadow DOM
    const template = document.getElementById("login-template");
    this.shadowRoot.appendChild(template.content.cloneNode(true));
    // A partir de aquí, SIEMPRE usar this.shadowRoot.getElementById()
    this.shadowRoot.getElementById("entrar").addEventListener("click", ...);
  }
}
customElements.define("login-acme", LoginAcme); // Registra el tag
```

ADVERTENCIA

ERROR FRECUENTE: Usar document.getElementById() dentro de un Web Component. Como el contenido está en el Shadow DOM, siempre debes usar this.shadowRoot.getElementById() o this.shadowRoot.querySelector(). document.getElementById() retornará null.

2.2 ES6 Módulos (import / export)

Los módulos ES6 permiten **dividir el código en archivos** separados con encapsulamiento. `auth.js` exporta funciones y `login.js` las importa. Para usar módulos en HTML es obligatorio declarar el script con `type="module"`.

```
// auth.js – exportar funciones individuales
export function validarUsuario(email, password) { ... }
export function crearSesion(usuario) { ... }

// login.js – importar solo las que necesita
import { validarUsuario, crearSesion } from "./auth.js";

// index.html – activar soporte de módulos
<script type="module" src="./login.js"></script>
```

TIP

Los módulos ES6 tienen scope propio: las variables declaradas en un módulo NO son globales. Además, los scripts de tipo module se ejecutan en modo estricto (strict mode) automáticamente.

2.3 Clases en JavaScript (OOP)

El proyecto usa dos patrones de clases. **ExamenStorage** usa métodos **static** (no necesita instanciarse). **GestorExamenes** usa el patrón instancia con **constructor** e inicialización en **init()**.

```
// Clase estática — se llama directamente sobre la clase, sin new
class ExamenStorage {
  static obtenerTodos() {
    const data = localStorage.getItem("examenes_data");
    return data ? JSON.parse(data) : [];
  }
  static guardarExamen(examen) { ... }
  static eliminarExamen(id) { ... }
}
// Uso: sin instanciar
const examenes = ExamenStorage.obtenerTodos();

// Clase instancia — se necesita "new"
class GestorExamenes {
  constructor() {
    this.examenActual = null; // estado de la instancia
    this.init();              // llamado al inicializar
  }
}
// Uso: instanciar en DOMContentLoaded
document.addEventListener("DOMContentLoaded", () => new GestorExamenes());
```

2.4 localStorage — CRUD Completo

localStorage es el mecanismo de persistencia del proyecto. Solo almacena **strings**, por eso se usa JSON.stringify() para guardar y JSON.parse() para leer.

Operación	Código	Descripción
CREATE / UPDATE	localStorage.setItem("clave", JSON.stringify(dato))	Guarda un objeto/array como string
READ	JSON.parse(localStorage.getItem("clave")) []	Lee y parsea. [] previene null
DELETE (item)	localStorage.removeItem("clave")	Elimina una clave específica
DELETE (todo)	localStorage.clear()	Borra todo el storage del origen

SECCIÓN 03

Desglose Práctico de Funciones

3.1 auth.js — Funciones de Autenticación

auth.js contiene toda la lógica de autenticación. Es el único archivo que lee y escribe en `localStorage["usuarios"]` y `localStorage["sesion"]`.

usuariosIniciales() — Inicialización con datos semilla

```
export function usuariosIniciales() {  
  // Solo crea el admin si NO existen usuarios todavía  
  let usuarios = localStorage.getItem("usuarios");  
  if (!usuarios) {  
    let usuarios = [  
      { nombre: "Administrador", email: "admin@acme.edu",  
        password: "admin123", cargo: "Administrativo" }  
    ];  
    localStorage.setItem("usuarios", JSON.stringify(usuarios));  
  }  
  // NOTA: si localStorage ya tiene datos, no hace nada  
}
```

ADVERTENCIA

BUG en el código original: dentro del if, se declara "let usuarios" de nuevo (shadowing). Esto crea una NUEVA variable local en lugar de reasignar la del scope exterior. Aunque funciona (`localStorage.setItem` recibe la variable interna), es una mala práctica que puede generar confusión.

validarUsuario() — Búsqueda con `.find()`

```
export function validarUsuario(email, password) {  
  let usuarios = JSON.parse(localStorage.getItem("usuarios")) || [];  
  // .find() devuelve el PRIMER elemento que cumple la condición  
  // o "undefined" si no encuentra ninguno  
  let usuario = usuarios.find(  
    (user) => user.email === email && user.password === password  
  );  
  return usuario; // undefined si no existe → el if falla  
}
```

registrarUsuario() — Prevención de duplicados


```
export function registrarUsuario(nombre, email, password) {
  let usuarios = JSON.parse(localStorage.getItem("usuarios")) || [];
  // Busca si ya existe un usuario con ese email
  let existe = usuarios.find((u) => u.email === email);
  if (existe) return false; // Indica fallo al llamador

  // Agrega el nuevo usuario y guarda todo el array
  usuarios.push({ nombre, email, password, cargo: "Estudiante" });
  localStorage.setItem("usuarios", JSON.stringify(usuarios));
  return true; // Indica éxito
}
```

TIP

El patrón de retornar true/false en funciones que modifican datos es muy limpio: el llamador puede reaccionar al resultado sin necesidad de try/catch. Es usado en registrarUsuario() y en guardarExamen().

3.2 login.js — El Web Component LoginAcme

Implementa el componente de login completo. Su método más complejo es el del botón "Entrar", que coordina validación, autenticación, feedback visual y redirección.

```
// Cuando el usuario hace click en "Entrar"
this.shadowRoot.getElementById("entrar").addEventListener("click", () => {

  // 1. Leer valores de los inputs del Shadow DOM
  let email    = this.shadowRoot.getElementById("email").value;
  let password = this.shadowRoot.getElementById("password").value;

  // 2. Validación básica: campos no vacíos
  let mensaje = this.shadowRoot.querySelector("#vista-login #mensaje");
  if (!email || !password) {
    mensaje.textContent = "Completa todos los campos.";
    mensaje.className = "error"; // Aplica estilos rojos via CSS
    return; // Detiene la ejecución
  }

  // 3. Delegar la autenticación al módulo auth.js
  let usuario = validarUsuario(email, password);

  // 4. Resultado y feedback visual
  if (usuario) {
    crearSesion(usuario);
    mensaje.textContent = "Bienvenido " + usuario.nombre;
    mensaje.className = "ok"; // Estilos verdes
    setTimeout(() => {
      window.location.href = "../PortalCreaciónExamen/CrearExamen.html";
    }, 1000); // Espera 1 segundo para que el usuario vea el mensaje
  } else {
    mensaje.textContent = "Email o contraseña incorrectos.";
    mensaje.className = "error";
  }
});
```

3.3 ExamenStorage — Clase Estática CRUD

ExamenStorage implementa el patrón Repository: abstrae toda la lógica de lectura/escritura del localStorage para los exámenes. Al ser estática, no necesita instanciarse.

guardarExamen() — Upsert (crear o actualizar)

```
static guardarExamen(examen) {
  const examenenes = this.obtenerTodos();
  // .findIndex() devuelve el índice del elemento o -1 si no existe
  const indice = examenenes.findIndex(e => e.id === examen.id);

  if (indice === -1) {
    // El examen NO existe → CREAR
    examen.id = Date.now(); // ID único basado en timestamp
    examen.codigo = this.generarCodigo(); // EX-001, EX-002...
    examen.fechaCreacion = new Date().toISOString();
    examenenes.push(examen);
  } else {
    // El examen YA existe → ACTUALIZAR (sobreescribir en el array)
    examen.fechaActualizacion = new Date().toISOString();
    examenenes[indice] = examen;
  }

  this.guardarTodos(examenenes); // Persiste todo el array
  return examen;
}
```

generarCodigo() — Códigos secuenciales

```
static generarCodigo() {
  const examenenes = this.obtenerTodos();
  const numero = examenenes.length + 1; // No es un ID real, solo un contador
  // padStart(3, "0") rellena con ceros: 1 → "001", 12 → "012"
  return `EX-${String(numero).padStart(3, "0")}`;
  // Resultado: "EX-001", "EX-002", ...
}
```

ADVERTENCIA

LIMITACIÓN: generarCodigo() usa examenenes.length + 1, no el ID del array. Si se elimina un examen, el próximo código puede repetirse. Ejemplo: hay EX-001 y EX-002, se elimina EX-001, el siguiente sería EX-002 de nuevo.

eliminarExamen() — Filtrado de arrays

```
static eliminarExamen(id) {  
  const examenes = this.obtenerTodos();  
  // .filter() devuelve un NUEVO array sin el elemento que tenga ese id  
  const filtrados = examenes.filter(e => e.id !== id);  
  this.guardarTodos(filtrados);  
  // El array original no se modifica; se reemplaza con el filtrado  
}
```

3.4 GestorExámenes — Gestión de UI (Event Delegation)

GestorExámenes es el controlador de la vista. Su método **manejarAccionesTabla()** implementa **Event Delegation**: en lugar de poner un listener en cada botón, pone uno en el contenedor y detecta qué botón fue clicado por su clase.

```
// Event Delegation — un solo listener maneja todos los botones de la tabla
document.getElementById("exámenes-tabla").addEventListener("click", (e) => {
  this.manejarAccionesTabla(e)
});

manejarAccionesTabla(e) {
  // e.target es el elemento exacto que recibió el click
  if (e.target.classList.contains("btn-editar-exam")) {
    // Leer el ID del examen guardado en data-id del botón
    const id = parseInt(e.target.getAttribute("data-id"));
    this.editarExamen(id);
  } else if (e.target.classList.contains("btn-eliminar-exam")) {
    const id = parseInt(e.target.getAttribute("data-id"));
    this.eliminarExamen(id);
  }
}

// En cargarExámenes(), los botones tienen el ID en data-id
`<button class="btn-editar-exam" data-id="${examen.id}">Editar</button>`
```

TIP

Event Delegation es eficiente porque: 1) Funciona con elementos creados dinámicamente (no existían cuando se cargó la página). 2) Solo necesita UN event listener para N botones. 3) Si se agregan más filas, no hay que añadir listeners nuevos.

agregarPregunta() — Inyección de HTML dinámico

```
agregarPregunta() {
  const preguntasList = document.getElementById("preguntas-list");
  // Calcular el número basado en elementos actuales
  const numPregunta = preguntasList.children.length + 1;

  // Template literal con HTML completo
  const preguntaHTML = `
    <div class="pregunta-item" id="pregunta-${numPregunta}">
      <textarea placeholder="Escribe la pregunta..."></textarea>
      <div id="respuestas-${numPregunta}">
        <input type="radio" name="correcta-${numPregunta}" value="0">
      </div>
    </div>`,

  // insertAdjacentHTML: más eficiente que innerHTML +=
  // "beforeend" inserta DENTRO del elemento, al FINAL
  preguntasList.insertAdjacentHTML("beforeend", preguntaHTML);

  // Los listeners deben añadirse DESPUÉS de insertar el HTML
  this.attachPreguntaEventListeners(numPregunta);
}
```

3.5 recolectarPreguntas() — Leer el DOM para construir datos

Esta función hace el camino inverso: **lee el DOM** (el HTML en pantalla) y construye el array de objetos que se guardará en localStorage. Es la función más compleja del módulo.

```
recolectarPreguntas() {  
  // 1. Seleccionar todos los bloques de pregunta  
  const preguntasElements = document  
    .getElementById("preguntas-list")  
    .querySelectorAll(".pregunta-item");  
  
  const preguntas = [];  
  
  preguntasElements.forEach((el, index) => {  
    // 2. Leer el texto del textarea de esta pregunta  
    const texto = el.querySelector(".pregunta-body textarea").value.trim();  
  
    // 3. Iterar sobre cada opción de respuesta  
    const respuestas = [];  
    let respuestaCorrecta = null;  
  
    el.querySelectorAll(".respuesta-item").forEach((respEl, respIndex) => {  
      const inputRespuesta = respEl.querySelector(".respuesta-input");  
      const radio = respEl.querySelector("input[type=radio]");  
  
      const textoResp = inputRespuesta.value.trim();  
      const esCorrecta = radio ? radio.checked : false;  
  
      if (textoResp) { // Solo incluir si tiene texto  
        respuestas.push({ id: respIndex, texto: textoResp });  
        if (esCorrecta) respuestaCorrecta = respIndex;  
      }  
    });  
  
    // 4. Solo agregar la pregunta si tiene texto Y respuestas  
    if (texto && respuestas.length > 0) {  
      preguntas.push({ id: index, texto, respuestas, respuestaCorrecta });  
    }  
  });  
  return preguntas;  
}
```

3.6 iniciarTemporizador() — setInterval y lógica de tiempo

El temporizador calcula el tiempo transcurrido comparando la hora actual con **fecha_inicio** guardada en localStorage, lo que lo hace resistente a recargas de página.

```
iniciarTemporizador(minutosTotales) {  
  // Calcular tiempo restante usando fecha de inicio guardada  
  const dataExamen = JSON.parse(localStorage.getItem("examen_actual"));  
  const ahora = new Date();  
  const inicio = new Date(dataExamen.fecha_inicio); // ISO string → Date  
  const transcurrido = Math.floor((ahora - inicio) / 1000); // en segundos  
  let tiempoRestante = (minutosTotales * 60) - transcurrido;  
  
  if (tiempoRestante <= 0) { this.finalizarExamen("Tiempo agotado"); return; }  
  
  // setInterval ejecuta la función cada 1000ms (1 segundo)  
  this.intervalo = setInterval(() => {  
    tiempoRestante--;  
    // Formatear: 75 seg → "01:15"  
    const min = Math.floor(tiempoRestante / 60);  
    const seg = tiempoRestante % 60;  
    display.textContent = `${String(min).padStart(2, "0")}:${String(seg).padStart(2, "0")}`;  
  
    // Alerta visual cuando queda menos de 1 minuto  
    display.style.color = tiempoRestante < 60 ? "#ef4444" : "";  
  
    if (tiempoRestante <= 0) {  
      clearInterval(this.intervalo); // SIEMPRE limpiar el intervalo  
      this.finalizarExamen("Tiempo agotado");  
    }  
  }, 1000);  
}
```

IMPORTANTE

Guardar el intervalo en `this.intervalo` es crucial. Permite llamar `clearInterval(this.intervalo)` para detenerlo cuando el examen termina. Sin esto, el intervalo seguiría ejecutándose indefinidamente (memory leak).

3.7 finalizarExamen() — Cálculo de resultados

Es el método central del módulo de resolución. Recoge respuestas del Shadow DOM, las compara con las correctas, calcula el porcentaje y renderiza el resultado.

```
finalizarExamen(motivo) {
  clearInterval(this.intervalo); // Detener el cronómetro

  // 1. Recolectar respuestas marcadas (radios checked)
  const inputs = this.shadowRoot.querySelectorAll("input[type=radio]:checked");
  const respuestasUsuario = {};

  inputs.forEach(input => {
    // input.name = "pregunta_0" → extraer el id de pregunta
    const idPregunta = input.name.replace("pregunta_", "");
    respuestasUsuario[idPregunta] = parseInt(input.value);
  });

  // 2. Comparar con respuestas correctas
  let correctas = 0;
  this.examenActual.preguntas.forEach((pregunta, i) => {
    const seleccionada = respuestasUsuario[pregunta.id ?? i];
    if (seleccionada !== undefined && seleccionada === pregunta.respuestaCorrecta) {
      correctas++;
    }
  });

  // 3. Calcular porcentaje y aprobar/reprobar
  const total = this.examenActual.preguntas.length;
  const porcentajeObtenido = Math.round((correctas / total) * 100);
  const aprobado = porcentajeObtenido >= this.examenActual.porcentaje;

  // 4. Guardar resultado y renderizar feedback
  this.guardarResultado(porcentajeObtenido, aprobado, correctas, total);
}
```

3.8 previewExamen.js — URLSearchParams

URLSearchParams permite leer los parámetros de la URL. En este proyecto, la URL lleva el código del examen seleccionado: **PreviewExamen.html?codigo=EX-001**

```
// URL: PreviewExamen.html?codigo=EX-001

// window.location.search = "?codigo=EX-001"
const params = new URLSearchParams(window.location.search);

// .get("codigo") extrae el valor del parámetro "codigo"
const codigoBuscado = params.get("codigo"); // "EX-001"

// Buscar el examen en el array usando ese código
const examen = examenes.find(e => e.codigo === codigoBuscado);

if (!examen) {
  contenedor.innerHTML = "<p>Examen no encontrado.</p>";
  return;
}

// En examenes.js, la URL se construye así al renderizar tarjetas:
`<a href="./PreviewExamen.html?codigo=${examen.codigo}">Resolver</a>`
```

SECCIÓN 04

Troubleshooting — Errores Comunes

Estos son los errores más frecuentes al implementar el tipo de código presente en este proyecto, con su causa raíz y solución.

ERROR

01

document.getElementById() retorna null dentro de un Web Component

Causa

El contenido del componente está en el Shadow DOM, que es un árbol DOM separado.

: document.getElementById() no puede ver dentro del Shadow DOM.

INCORRECTO

```
document.getElementById("entrar").addEventListener(...)
```

CORRECTO

```
this.shadowRoot.getElementById("entrar").addEventListener(...)
```

Tip

:

Siempre usa this.shadowRoot como punto de inicio para cualquier consulta dentro del componente.

ERROR

02

JSON.parse() falla con null (localStorage vacío)

Causa

localStorage.getItem() devuelve null si la clave no existe. JSON.parse(null) lanza SyntaxError.

:

INCORRECTO

```
const data = JSON.parse(localStorage.getItem("clave"));
```

CORRECTO

```
const data = JSON.parse(localStorage.getItem("clave")) || [];
```

Tip

:

El operador || [] provee un valor por defecto cuando el resultado es null/undefined/falsy.

ERROR

03

Los botones añadidos dinámicamente no responden a eventos

Causa

Los event listeners se añaden ANTES de que el elemento exista en el DOM. En el momento de

: addEventListener, el botón aún no ha sido insertado.

INCORRECTO

```
// Añadir listener y luego insertar HTML
```

CORRECTO

```
preguntasList.insertAdjacentHTML("beforeend", preguntaHTML); this.attachPreguntaEventListeners(numPregunta); // DESPUÉS del insert
```

Tip

:

Siempre añada listeners DESPUÉS de insertar el HTML, o usa Event Delegation en el contenedor padre.

ERROR

04

El temporizador no se detiene (setInterval sin clearInterval)

Causa

Si no se guarda la referencia del intervalo, no se puede detener. El intervalo seguirá ejecutándose

: aunque la pantalla cambie.

INCORRECTO	CORRECTO
<pre>setInterval(() => { ... }, 1000); // Sin guardar la referencia</pre>	<pre>this.intervalo = setInterval(() => { ... }, 1000); // Al terminar: clearInterval(this.intervalo);</pre>
Tip : Siempre guarda el return value de setInterval/setTimeout para poder cancelarlo.	

ERROR 05 Scripts de tipo module no funcionan abriendo el HTML directamente	
Causa : Los ES Modules requieren un servidor HTTP. Al abrir index.html como file:// en el navegador, la política CORS bloquea los imports.	
INCORRECTO	CORRECTO
<pre>// Abrir directamente: file:///ruta/index.html (falla CORS)</pre>	<pre>// Usar servidor local: npx serve . ó Live Server (extensión de VS Code)</pre>
Tip : Este es el error más frecuente al empezar con ES Modules. Siempre usa un servidor local.	

ERROR 06 Los radio buttons de diferentes preguntas se interfieren	
Causa : Los radio buttons se agrupan por el atributo "name". Si dos preguntas tienen el mismo name, seleccionar una opción deselecta la de la otra pregunta.	
INCORRECTO	CORRECTO
<pre>// Misma pregunta y diferente</pre>	<pre>` ` // Cada pregunta tiene su propio grupo: correcta-1, correcta-2...</pre>
Tip : Los IDs dinámicos (correcta-1, correcta-2) son el patrón correcto. El método reenumerarPreguntas() actualiza estos nombres cuando se elimina una pregunta.	

SECCIÓN 05

Simulacro de Examen

Las siguientes 5 preguntas cubren los temas más probables de evaluación sobre este proyecto. Incluyen preguntas teóricas, de análisis de código y de completar código.

PREGUNTA 1 [TEÓRICA]

¿Cuál es la diferencia entre `document.getElementById()` y `this.shadowRoot.getElementById()`, y cuándo debe usarse cada uno en este proyecto?

- A No hay diferencia; ambos buscan en todo el documento.
- B `document` busca en el DOM principal; `shadowRoot` busca dentro del Shadow DOM del componente. Se usa `document` en `app.js` y `shadowRoot` en `login.js` y `RealizarExamen.js`.
- C `shadowRoot` solo funciona en Firefox; `document` es cross-browser.
- D `this.shadowRoot.getElementById()` es más lento porque recorre más nodos.

Respuesta correcta: B

Explicación: Los Web Components en este proyecto usan Shadow DOM. Todo el contenido del componente vive en un árbol DOM aislado accesible solo por `this.shadowRoot`. `document.getElementById()` no puede ver ese árbol. En `GestorExámenes` (`app.js`) no hay Shadow DOM, por lo que se usa `document.getElementById()` directamente.

PREGUNTA 2 [ANÁLISIS DE CÓDIGO]

Observa el siguiente fragmento de `ExamenStorage`. ¿Qué devuelve `guardarExamen()` cuando se llama con un examen que YA existe en `localStorage` (su `id` ya está en el array)?

```
static guardarExamen(examen) {
  const examenes = this.obtenerTodos();
  const indice = examenes.findIndex(e => e.id === examen.id);
  if (indice === -1) {
    examen.id = Date.now();
    examenes.push(examen);
  } else {
    examen.fechaActualizacion = new Date().toISOString();
    examenes[indice] = examen;
  }
  this.guardarTodos(examenes);
  return examen;
}
```

- A Retorna -1 para indicar que el examen ya existía.
- B Retorna `undefined` porque el `else` no tiene `return`.
- C Retorna el objeto examen actualizado con la nueva `fechaActualizacion`.
- D Lanza un Error porque no puede sobrescribir un examen existente.

Respuesta correcta: C

Explicación: Cuando índice !== -1, se entra al bloque else: se asigna fechaActualizacion al objeto examen, se sobrescribe en el array con examenes[indice] = examen, se persiste con guardarTodos(), y finalmente se retorna el mismo objeto examen (ahora con fechaActualizacion actualizada). El return está fuera del if/else.

PREGUNTA 3 [COMPLETAR CÓDIGO]

El siguiente código tiene UN error que impide que los exámenes se muestren correctamente en Exámenes.html. Identifica cuál es el error y cómo corregirlo:

```
// examenes.js
function renderizarExámenes() {
  const examenes = obtenerExámenes();
  const contenedor = document.querySelector(".Contenedor-Exámenes");

  contenedor.innerHTML = examenes.map(examen => (`
    <div class="Exámenes">
      <h1>${examen.titulo}</h1>
      <a href="./PreviewExamen.html?id=${examen.codigo}">
        Resolver Examen
      </a>
    </div>
  `)).join("");
}

renderizarExámenes(); // Llamado directo, sin DOMContentLoaded
```

- A** El parámetro de la URL debería ser ?codigo= en lugar de ?id=, y la función debería llamarse dentro de DOMContentLoaded.
- B** Se debe usar document.getElementById en lugar de querySelector.
- C** El .map() debe ser reemplazado por un .forEach() para funcionar correctamente.
- D** La URL debe apuntar a RealizarExamen.html, no a PreviewExamen.html.

Respuesta correcta: A

Explicación: Hay dos errores: 1) En previewExamen.js se lee params.get("codigo"), pero aquí la URL genera ?id= en vez de ?codigo=, por lo que el examen no se encontraría. 2) Llamar renderizarExámenes() directamente (sin DOMContentLoaded) puede ejecutarse antes de que el DOM esté listo, haciendo que contenedor sea null.

PREGUNTA 4 [TEÓRICO-PRÁCTICA]

¿Por qué el método renumerarPreguntas() actualiza el atributo name de los radio buttons al eliminar una pregunta? ¿Qué problema específico previene?

- A** Para cambiar el diseño visual de las opciones de respuesta.
- B** Para que el HTML sea válido semánticamente según el estándar W3C.
- C** Para evitar que radio buttons de diferentes preguntas compartan el mismo name, lo que haría que seleccionar una opción deseleccione opciones de otras preguntas y que recolectarPreguntas() mapee las respuestas al grupo incorrecto.

D Porque localStorage solo permite almacenar datos con nombres únicos.

Respuesta correcta: C

Explicación: Los radio buttons con el mismo atributo name forman un grupo exclusivo: solo uno puede estar marcado a la vez dentro del grupo. Si la "Pregunta 2" se elimina y las preguntas no se reenumeran, la "Pregunta 3" (ahora la segunda) mantendría name="correcta-3". Esto es inconsistente y además recolectarPreguntas() usa querySelectorAll para leer los radios por su contenedor, con lo que IDs mal alineados corrompen la estructura de datos guardada.

PREGUNTA 5 [PRÁCTICA]

Escribe el código JavaScript para una función obtenerUsuarioActual() que lea la sesión guardada en localStorage["sesion"] y retorne el objeto del usuario. Si no hay sesión activa, debe retornar null. Luego explica dónde y cómo se usaría en este proyecto.

```
// Tu solución aquí:
function obtenerUsuarioActual() {
  // ...
}
```

Explicación: SOLUCIÓN ESPERADA: function obtenerUsuarioActual() { const sesion = localStorage.getItem("sesion"); return sesion ? JSON.parse(sesion) : null; } USO EN EL PROYECTO: Se llamaría en CrearExamen.html al cargar la página para verificar que el usuario está autenticado (si retorna null, redirigir al login). También en el header para mostrar el nombre del usuario activo. auth.js ya tiene crearSesion() que guarda en "sesion", esta función complementa el ciclo de autenticación.

RESUMEN DE TEMAS CLAVE

- Web Components: HTMLElement, attachShadow, connectedCallback, customElements.define
- Shadow DOM: this.shadowRoot vs document — siempre shadowRoot dentro del componente
- ES6 Modules: export / import — requiere type="module" en el script y servidor local
- localStorage: setItem + JSON.stringify / getItem + JSON.parse || [] / removeItem
- Clases: static (ExamenStorage) vs instancia (GestorExamenes) — constructor + init()
- Array Methods: .find(), .findIndex(), .filter(), .map(), .forEach(), .every()
- Event Delegation: listener en el padre, detectar e.target.classList por botón
- insertAdjacentHTML("beforeend", html) — más eficiente que innerHTML +=
- setInterval / clearInterval — siempre guardar la referencia del intervalo
- URLSearchParams: new URLSearchParams(window.location.search).get("clave")
- Date: Date.now() para IDs, new Date().toISOString() para timestamps
- padStart(n, "0"): formatear números con ceros a la izquierda — "01:05"